

MODELAMIENTO DE BASES DE DATOS

UNIDAD N° II

Construyendo un Modelo Relacional Normalizado



SEMANA 4

Consideraciones previas

Alineación Curricular del Material de Estudio

El contenido que se expone a continuación está ligado a la siguiente unidad de competencia:

- Construir un modelo Relacional que responda a las necesidades de información transaccional del negocio, de acuerdo a requerimientos específicos.

Sobre las fuentes utilizadas en el material

El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.

Introducción

El modelo relacional, para el modelado y la gestión de bases de datos, es un modelo de datos basado en la lógica de predicados y en la teoría de conjuntos.

Tras ser postuladas sus bases en 1970 por Edgar Frank Codd, de los laboratorios IBM en San José (California), no tardó en consolidarse como un nuevo paradigma en los modelos de base de datos.

Su idea fundamental es el uso de relaciones. Estas relaciones podrían considerarse en forma lógica como conjuntos de datos llamados tuplas. Pese a que esta es la teoría de las bases de datos relacionales creadas por Codd, la mayoría de las veces se conceptualiza de una manera más fácil de imaginar, pensando en cada relación como si fuese una tabla que está compuesta por registros (cada fila de la tabla sería un registro o “tupla”) y columnas (también llamadas “campos”).

Es el modelo más utilizado en la actualidad para modelar problemas reales y administrar datos dinámicamente.

El modelo relacional desarrolla un esquema de base de datos (data base schema) a partir del cual se podrá realizar el modelo físico o de implementación en el DBMS.

Este modelo está basado en que todos los datos están almacenados en tablas (entidades/relaciones) y cada una de estas es un conjunto de datos, por tanto, una base de datos es un conjunto de relaciones. La agrupación se origina en la tabla: tabla -> fila (tupla) -> campo (atributo)

El Modelo Relacional se ocupa de:

- La estructura de datos.
- La manipulación de datos.
- La integridad de los datos.

Donde las relaciones están formadas por:

- Atributos (columnas).
- Tuplas (Conjunto de filas).

Ideas fuerza

- Conceptos del Modelo Relacional: Veremos qué es el modelo lógico de una base de datos, su objetivo, sus características, su terminología, las 12 reglas de Codd y claves y restricciones de este modelo.
- Transformación del Modelo Entidad Relación al Modelo Relacional: Veremos cómo llevar a cabo la transformación manual y automática de un Modelo Entidad Relación previamente normalizado al Modelo Relacional. Para esto veremos cómo se transforman las entidades fuertes, las entidades débiles, las relaciones 1:1, 1:N, M:N, relaciones recursivas y cuando hay jerarquías de entidades (relaciones de supertipo/subtipos).
- Construcción del Modelo Relacional: Veremos cómo construir este modelo en la herramienta SQL Data Modeler.

Índice

Introducción	3
Ideas fuerza	4
Diseño Lógico	7
Modelo Relacional	8
Objetivos del Modelo Relacional	9
Características del Modelo Relacional	10
Las 12 Reglas de Codd	12
Terminología del Modelo Relacional	14
Relación o Tabla	15
Atributos.....	16
Dominio	16
Tupla, Grado y Cardinalidad.....	17
Claves	18
Restricciones de las Relaciones	19
Tipos de datos para los Atributos.....	21
Transformación del Modelo Entidad Relación Normalizado al Modelo Relacional .	23
Transformación de Entidad-Fuerte.....	23
Transformación de las relaciones 1:N	24
Transformación de las relaciones M:N	25
Transformación de las relaciones 1:1	26
Transformación de las relaciones recursivas	27
Transformación de Entidades Débiles	28
Transformación de Relaciones de Exclusividad	29
Transformación de Relaciones de Jerarquías	30
1. Diseño de los subtipos y supertipo en una sola Relación.....	30
2. Diseño de los Subtipos en Relaciones separadas	31
3. Diseño del supertipo y los subtipos en Relaciones separadas	32
Construcción del Modelo Relacional Gráfico y No Gráfico Caso 1.....	34

<i>Solución Caso 1 Modelo Relacional Gráfico</i>	<i>35</i>
<i>Solución Caso 1 Modelo Relacional No Gráfico</i>	<i>44</i>
<i>Conclusión</i>	<i>51</i>
<i>Referencias.....</i>	<i>52</i>

Desarrollo

Diseño Lógico

El objetivo del diseño lógico es convertir los esquemas conceptuales en un esquema lógico que se ajuste al modelo del SGBD (Sistema Gestor de Bases de Datos) sobre el que se vaya a implementar el sistema.

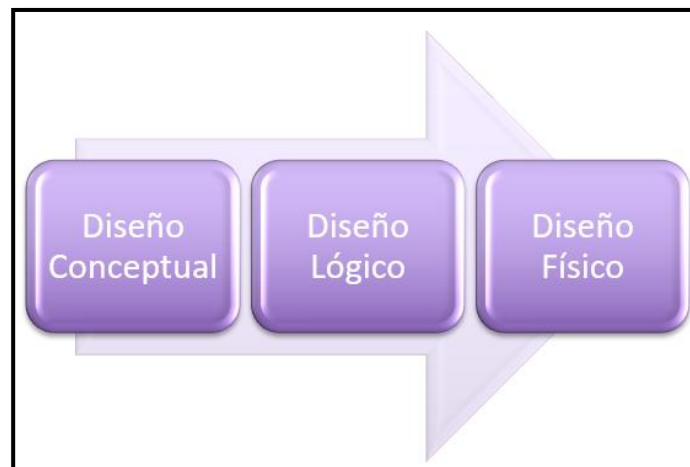
Mientras que el objetivo fundamental del diseño conceptual es representar un conjunto de conceptos de una realidad, el objetivo del diseño lógico es obtener una representación que use, del modo más eficiente posible, los recursos que el modelo de SGBD posee para estructurar los datos y para modelar las restricciones.

El Modelo Lógico utilizado para SGBD Relacionales es el **Modelo Relacional**.

En general, los diseñadores y administradores de bases de datos relacionales usan esquemas conceptuales Entidad-Relación porque se adaptan muy bien para ser transformados al Modelo Relacional.

Objetivo del diseño lógico:

- Convertir los esquemas conceptuales en un esquema lógico que se ajuste al modelo del SGBD:

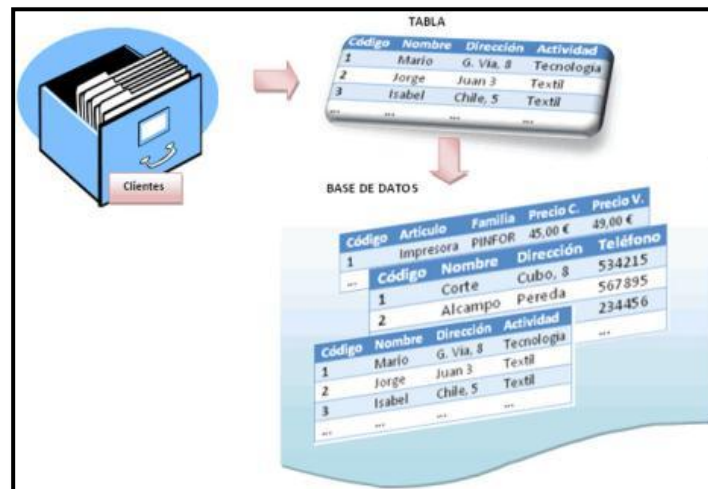


Modelo Relacional

En 1970, el modo en que se veían las bases de datos cambió por completo cuando Edgar Frank Codd introdujo el Modelo Relacional.

En aquellos momentos, el enfoque existente para la estructura de las bases de datos que se basaban en el modelo de red y el modelo jerárquico, los dos modelos lógicos que constituyeron la primera generación de los SGBD.

Codd demostró que esta primera generación de bases de datos limitaba en gran medida los tipos de operaciones que los usuarios podían realizar sobre los datos. Además, estas bases de datos eran muy vulnerables a cambios en el entorno físico.

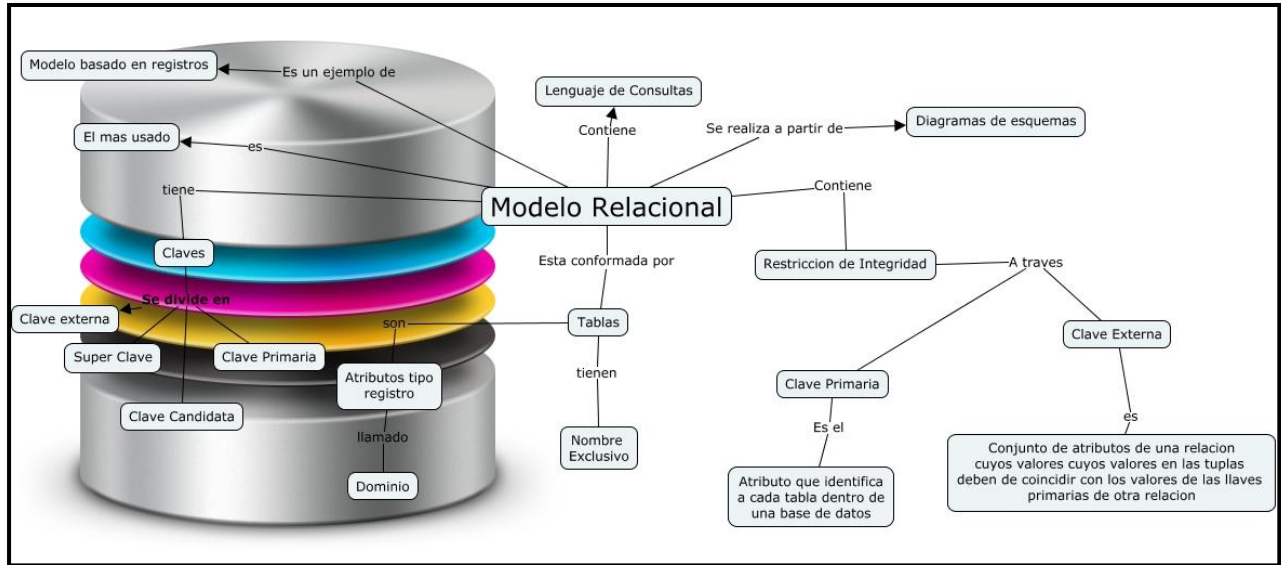


El **Modelo Relacional** representa la segunda generación de los SGBD. En él, todos los datos están estructurados a nivel lógico para ser convertidos posteriormente en tablas formadas por filas y columnas.

La ventaja del modelo relacional es que los datos se representan conceptualmente como se almacenarán de un modo en que los usuarios entienden con mayor facilidad.

El Modelo Relacional se ocupa de tres aspectos principales de la información: la estructura de datos, la manipulación de datos y la integridad de los datos.

Desde el enfoque relacional, los datos se organizan en estructuras llamadas Relaciones, cada una de las cuales están formadas por Atributos (columnas) y un conjunto de filas llamadas Tuplas. Así, una relación se compone de una colección de tuplas cuyas propiedades están descritas por cierto número de atributos.



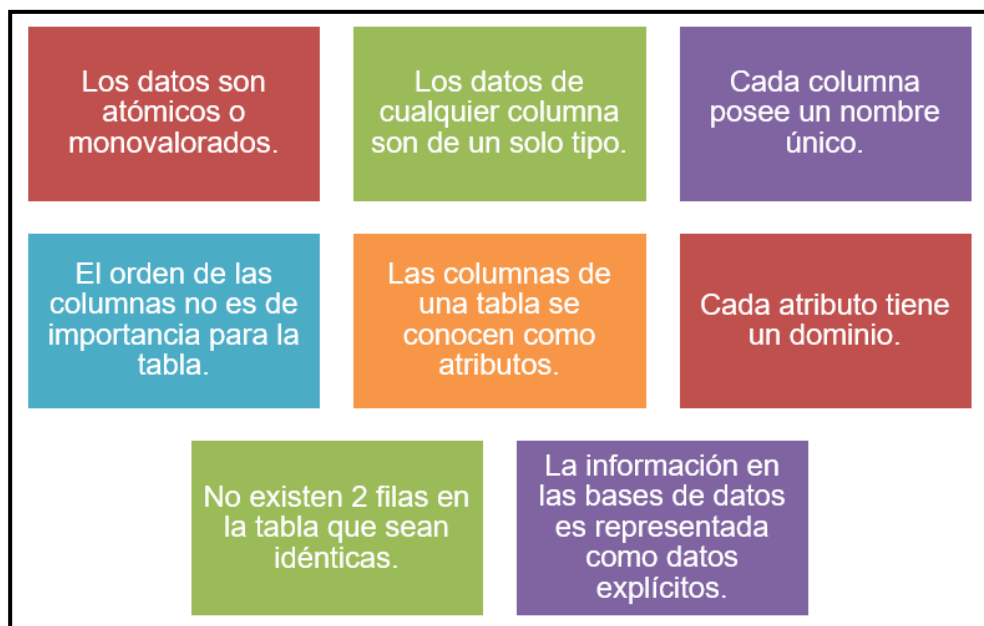
Objetivos del Modelo Relacional



Los objetivos que este modelo persigue son:

- Independencia Física: La forma de almacenar los datos no debe influir en su manipulación. Si el almacenamiento físico cambia, los usuarios que acceden a esos datos no tienen que modificar sus aplicaciones.
- Independencia Lógica: Las aplicaciones que utilizan la base de datos no deben ser modificadas por que se inserten, actualicen y eliminen datos.
- Flexibilidad: En el sentido de poder presentar a cada usuario los datos de la forma en que éste prefiera.
- Uniformidad: Las estructuras lógicas de los datos siempre tienen una única forma conceptual (las tablas), lo que facilita la creación y manipulación de la base de datos por parte de los usuarios.
- Sencillez: Las características anteriores hacen que este Modelo sea fácil de comprender y de utilizar por parte del usuario final.

Características del Modelo Relacional



Las características más importantes de los Modelos Relacionales son:

- Los datos en la tabla tienen un solo valor, es decir son atómicos o monovalorados; no se admiten valores múltiples, por lo tanto una columna tiene un solo valor, nunca un conjunto de valores (1FN).
- Los datos de cualquier columna son de un solo tipo. Por ejemplo, una columna puede contener nombres de clientes, y en otra puede tener fechas de nacimiento.
- Cada columna posee un nombre único.
- El orden de las columnas no es de importancia para la tabla.
- Las columnas de una relación se conocen como atributos. Cada atributo tiene un dominio, que es una descripción física y lógica de valores permitidos.
- No existen 2 filas en la tabla que sean idénticas.
- La información en las bases de datos es representada como datos explícitos.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

¿Cuál/es es/son las razones que explican la buena capacidad de adaptabilidad de los esquemas conceptuales Entidad-Relación para ser transformados al Modelo Relacional?



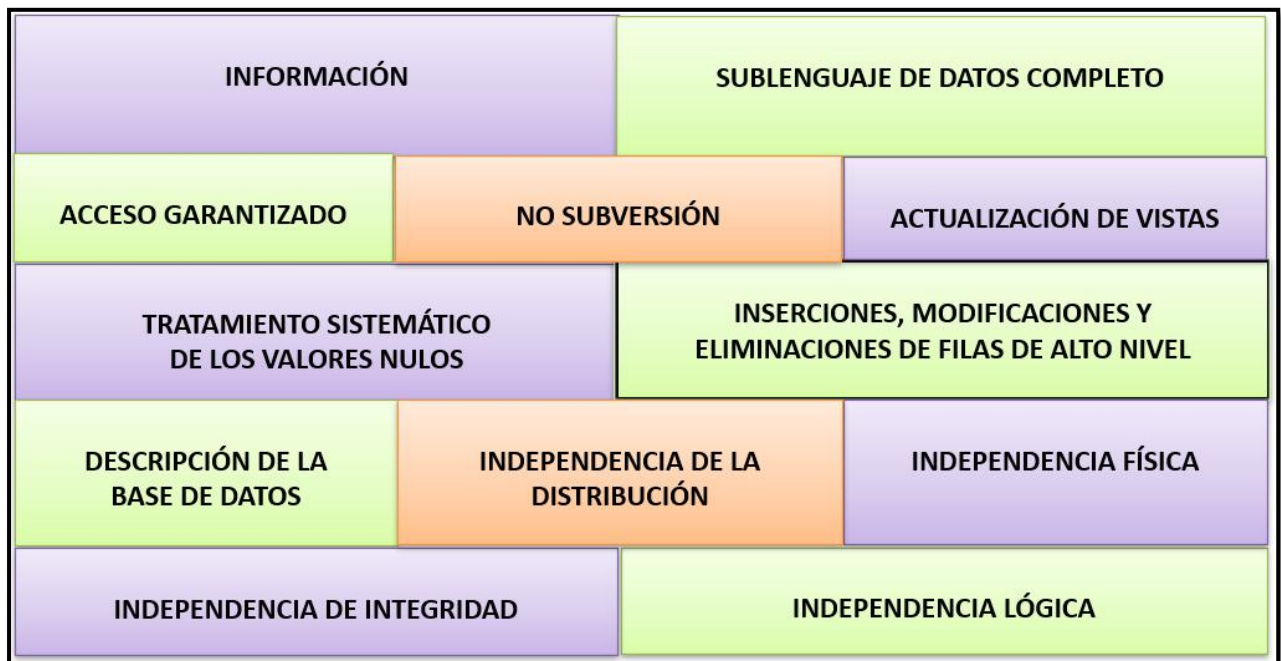
Las 12 Reglas de Codd

Preocupado por los productos que decían ser Sistemas Gestores de Bases de Datos Relacionales (**RDBMS o SGBDR**) sin serlo, Codd publica las 12 reglas que debe cumplir todo RDBMS para ser considerado Relacional.

Estas reglas en la práctica las cumplen pocos sistemas relacionales en su totalidad, pero una BD Relacional debería cumplir con el mayor número de ellas. Las reglas y sus significados son:

- **Información:** Toda la información de la Base de Datos (Metadatos) debe estar representada explícitamente en el esquema lógico. Es decir, todos los datos están en las tablas.
- **Acceso Garantizado:** Todo dato es accesible sabiendo el valor de su clave y el nombre de la columna o atributo que contiene el dato.
- **Tratamiento Sistemático De Los Valores Nulos:** El SGBD debe permitir el tratamiento adecuado de estos valores. Así, el valor NULO se utiliza para representar la ausencia de información de un determinado registro en un atributo concreto.
- **Descripción De La Base De Datos:** La descripción de la base de datos es almacenada de la misma manera que los datos ordinarios, esto es, en tablas y columnas, y debe ser accesible a los usuarios autorizados.
- **Sublenguaje De Datos Completo:** Al menos debe de existir un lenguaje que permita el manejo completo de la base de datos. Este lenguaje, por lo tanto, debe permitir realizar cualquier operación sobre la misma.
- **Actualización De Vistas:** El SGBD debe encargarse de que las vistas (objetos que permitan mostrar información específica para cada usuario) muestren la última información.
- **Inserciones, Modificaciones Y Eliminaciones De Filas De Alto Nivel:** Cualquier operación de modificación debe actuar sobre un conjuntos de filas.
- **Independencia Física:** Los datos deben de ser accesibles desde la lógica de la base de datos aun cuando se modifique el almacenamiento. La forma de acceder a los datos no varía porque el esquema físico de la base de datos cambia.
- **Independencia Lógica:** Los programas no deben verse afectados por cambios en las tablas. Que las tablas cambien no implica que cambien los programas.

- **Independencia De Integridad:** Las restricciones de integridad deben estar separadas de los programas, almacenadas en el catálogo de la BD para ser editadas mediante un sublenguaje de datos.
- **Independencia De La Distribución:** Las aplicaciones no deben verse afectadas al distribuir (dividir entre varias máquinas), o al cambiar la distribución ya existente de la Base de Datos.
- **No Subversión:** Si el SGBD posee un lenguaje de bajo nivel que permite el recorrido fila a fila, éste no se puede utilizar para incumplir o evitar las reglas relacionales establecidas por el lenguaje de datos de alto nivel.

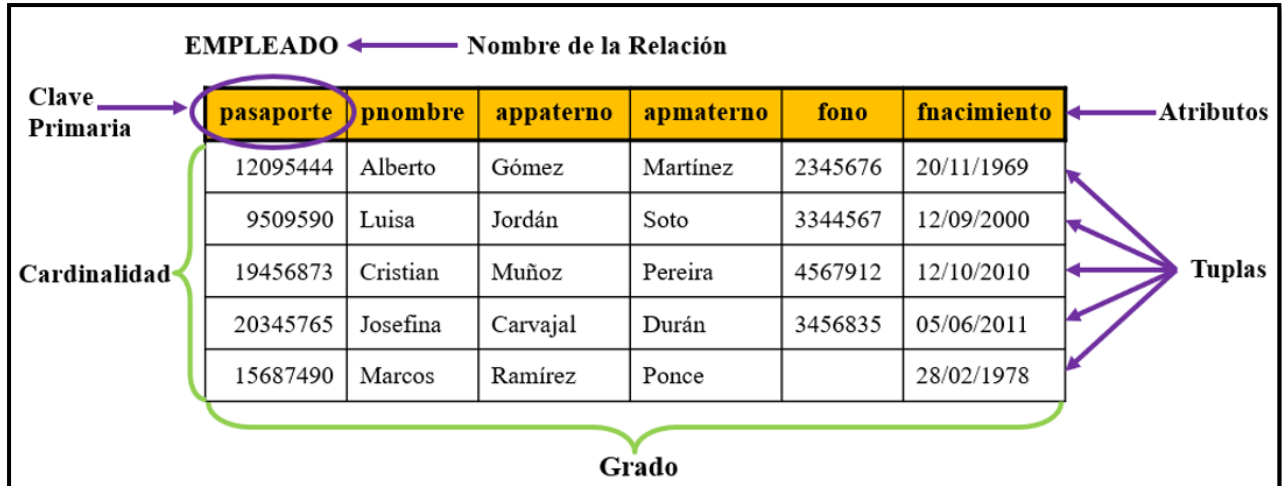


Terminología del Modelo Relacional

Los términos más importantes del Modelo Relacional son:

- Relación: corresponde con la idea general de tabla.
- Tupla: corresponde con una fila.
- Atributo: corresponde con una columna.
- Cardinalidad: número de tuplas (m).
- Grado: número de atributos (n).
- Dominio: colección de valores de los cuales el atributo obtiene su atributo.
- Clave Primaria: identificador único de una tupla.

Terminología Relacional		Terminología de Tablas
Relación	=	Tabla
Tupla	=	Fila
Atributo	=	Columna
Grado	=	Número de columnas
Cardinalidad	=	Número de filas



Relación o Tabla

Según el Modelo Relacional el elemento fundamental es lo que se conoce como Relación, aunque más habitualmente se le llama Tabla.

La base de datos es vista como una colección de relaciones.

Una relación puede ser vista como una tabla, con filas llamadas tuplas y con cabecera de columnas llamadas atributos.

Las relaciones tienen las siguientes características:

- Cada relación tiene un nombre específico y diferente al resto de las relaciones.
- Los valores de los atributos son atómicos: en cada tupla, cada atributo (columna) toma un solo valor. Se dice que las relaciones están normalizadas.
- No hay dos atributos que se llamen igual.
- El orden de los atributos no importa: los atributos no están ordenados.
- Cada tupla es distinta de las demás: no hay tuplas duplicadas
- El orden de las tuplas no importa: las tuplas no están ordenadas.

Nombre Relación o Tabla						
atributo 1	atributo 2	atributo 3	atributo 4		atributo n	
valor 1,1	valor 1,2	valor 1,3	valor 1,4		valor 1,n	← tupla 1
valor 2,1	valor 2,2	valor 2,3	valor 2,4		valor 2,n	← tupla 2
valor 3,1	valor 3,2	valor 3,3	valor 3,4		valor 3,n	← tupla 3
.....						←
valor m,1	valor m,2	valor m,3	valor m,4		valor m,n	← tupla m

Atributos

Un Atributo en el Modelo Relacional representa una propiedad que posee esa Relación y equivale al atributo del Modelo E-R.

Se corresponde con la idea de campo o columna.

En el caso de que sean varios los atributos de una misma tabla, definidos sobre el mismo dominio, habrá que darles nombres distintos, ya que una tabla no puede tener dos atributos con el mismo nombre.

Por ejemplo, la información de las oficinas de una empresa inmobiliaria se representa mediante la relación OFICINA, que tiene columnas para los atributos n°oficina (número de oficina), calle, área, teléfono y fax.

n°oficina	calle	area	telefono	fax
100	Lyon 2345	Las Condes	964201240	964201340
110	Alameda 234	Santiago Centro	964215760	964215670
120	Luis Thayer Ojeda	Providencia	964520250	964520255
130	Baldomero Lillo 2345	Puente Alto	964284440	
140	Calle Crucero 3456	La Dehesa	965678904	964252811

Dominio

El dominio dentro de la estructura del Modelo Relacional es el conjunto de valores que puede tomar un atributo.

- Un dominio contiene todos los posibles valores que puede tomar un determinado atributo. Dos atributos distintos pueden tener el mismo dominio.
- Un dominio es un conjunto finito de valores del mismo tipo.
- Los dominios poseen un nombre para poder referirnos a él y así poder ser reutilizable en más de un atributo.

En el ejemplo, la tabla muestra los dominios de los atributos de la relación OFICINA. Nótese que en esta relación hay dos atributos que están definidos sobre el mismo dominio: teléfono y fax.

Atributo	Nombre del Dominio	Descripción	Definición
nº oficina	NUM_OFICINA	Posibles valores de número de oficina	3 caracteres, rango 100 - 990
calle	NOM_CALLE	Nombres de calles y número de Santiago donde se ubica la oficina	25 caracteres
area	NOM_AREA	Área de Santiago en la que se encuentra ubicada la oficina	20 caracteres
telefono	NUM_TEL_FAX	Números de teléfono de Santiago	9 caracteres
fax	NUM_TEL_FAX	Números de teléfono de Santiago	9 caracteres

Tupla, Grado y Cardinalidad

- Tupla: es cada una de las filas de la relación. Representa por tanto el conjunto de cada elemento individual (ejemplar u ocurrencia) de esa tabla. En la relación OFICINA, cada tupla tiene cinco valores, uno para cada atributo. Las tuplas de una relación no siguen ningún orden.
- Grado: número de columnas de la relación (número de atributos). La relación OFICINA es de grado cinco porque tiene cinco atributos. Esto quiere decir que cada fila de la tabla es una tupla con cinco valores.

- Cardinalidad: número de tuplas de una relación (número de filas). Ya que en las relaciones se van insertando y borrando tuplas a menudo, la cardinalidad de estas varía constantemente.

Cardinalidad	pasaporte	pnombre	appaterno	apmaterno	fono	fnacimiento	Tuplas
	12095444	Alberto	Gómez	Martínez	2345676	20/11/1969	
	9509590	Luisa	Jordán	Soto	3344567	12/09/2000	
	19456873	Cristian	Muñoz	Pereira	4567912	12/10/2010	
	20345765	Josefina	Carvajal	Durán	3456835	05/06/2011	
	15687490	Marcos	Ramírez	Ponce		28/02/1978	
Grado							

Claves

Ya que en una relación no hay tuplas repetidas, éstas se pueden distinguir unas de otras, es decir, se pueden identificar de modo único. La forma de identificarlas es mediante los valores de sus atributos.

- Clave primaria: clave candidata que se escoge como identificador de las tuplas. Se elige primaria la candidata que identifique mejor a cada tupla en el contexto de la base de datos. Por ejemplo, un atributo con el RUT sería clave candidata de una tabla de clientes, aunque si en esa relación existe un atributo de código de cliente,

este sería mejor candidato para clave principal, porque es mejor identificador para ese contexto.

- Clave alternativa: Cualquier clave candidata que no sea primaria y que también puede identificar de manera única una tupla. Al momento de crear la relación como tabla en la Base de Datos se debe definir una constraints de tipo UNIQUE.
- Clave candidata: conjunto de atributos que identifican en forma única cada tupla de la relación. Es decir, columnas cuyos valores no se repiten para esa tabla.
- Clave externa, ajena o foránea: atributo cuyos valores coinciden con una clave candidata (normalmente primaria) de otra tabla.

Restricciones de las Relaciones

Una restricción es una condición que deben cumplir los datos de la base de datos.

Las restricciones propias y que son definidas por el hecho de que la base de datos es relacional son:

- No puede haber dos tuplas iguales.
- El orden de las tuplas no es significativo.
- El orden de los atributos no es significativo.
- Cada atributo sólo puede tomar un valor en el dominio en el que está inscrito.

Las restricciones incorporadas por los usuarios son:

- **Clave primaria (PRIMARY KEY):** hace que los atributos marcados como clave primaria no puedan tener valores repetidos. Además, obliga a que esos atributos deben tener un valor. Si la clave primaria la forman varios atributos, ninguno de ellos podrá estar vacío.
- **Unicidad (UNIQUE):** impide que los valores de los atributos marcados de esta forma puedan repetirse. Esta restricción debe indicarse en todas las claves alternativas.

- **Obligatoriedad (NOT NULL):** obliga que el atributo marcado de esta forma debe tener un valor, es decir impide que pueda contener el valor nulo (**NULL**).
- **Integridad referencial (FOREIGN KEY):** sirve para indicar una clave externa o foránea. Cuando una clave se marca con integridad referencial no se podrán introducir valores que no estén incluidos en los atributos relacionados con esa clave de la relación “padre”. Esto último causa problemas en las operaciones de borrado y modificación de registros, ya que si se ejecutan esas operaciones sobre la tabla principal quedarán filas en la tabla secundaria con la clave externa sin integridad.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

¿Cuál es el impacto que adquiere la existencia de restricciones en las relaciones para un usuario, cuando interactúa frente a una base de datos?



Tipos de datos para los Atributos

Tipo de Dato	Descripción
VARCHAR2(tamaño)	Tipo de dato de caracteres de longitud variable. Se debe especificar su tamaño. Tamaño mínimo: 1, tamaño máximo: 4000.
CHAR[(tamaño)]	Tipo de dato de caracteres de longitud fija. Su tamaño mínimo es 1 y el máximo es 2000.
NUMBER(precisión,escala)	Tipo de dato numérico de longitud variable. La precisión indica el largo total y la escala los decimales. Si no se indica la escala, el número será un valor entero. El rango para la precisión va desde 1 a 38. El rango para la escala va desde -84 to 127.
DATE	Valores de fecha y hora.
LONG	Dato de caracteres de longitud variable de hasta 2 gigabytes.

Tipo de Dato	Descripción
CLOB	Dato de caracteres de longitud variable de hasta 4 gigabytes.
BLOB	Dato binario de hasta 4 gigabytes.
BFILE	Dato binario almacenado fuera de la BD; de hasta 4 gigabytes.
TIMESTAMP [(precisión)]	Es una extensión de DATE, almacena año, mes, día, hora, minuto, segundo y fracción de segundo. La precisión indica el número de dígitos para la fracción de segundos. El rango es de 0 a 9 y el valor por defecto es 6.

Propiedades de Entidad - Entity_1

Atributos

Identificadores Únicos

Subtipos

Propiedades de Volumen

Realizar Ingeniería a

Comentarios

Comentarios en RDBMS

Atributos Solapados

Notas

Análisis de Impacto

Medidas

Solicitudes de Cambio

Partes Responsables

Documentos

Propiedades Dinámicas

Propiedades Definidas por el Usuario

Tipos de Clasificación

Resumen

Atributos

Detalles Descripción General UDP

Atributos:

	Nombre	Tipo de Dato
1	IdAlumno	Integer
2	RutAlumno	Integer
3	DvAlumno	VARCHAR (2)
4	FechaNacimiento	Date
5	Direccion	VARCHAR (50)

Propiedades de Atributo

Nombre: IdAlumno

Tipo de Dato: ☐ Dominio ☒ Lógico ☐ Distinto

☐ Estructurado ☐ Recopilación

Tipo de Origen: Integer Preferencia ☐

☒ UID Primario ☐ UID de Relación ☒ Obligatorio ☐ Anticuoado

Comentarios Comentarios en RDBMS Notas

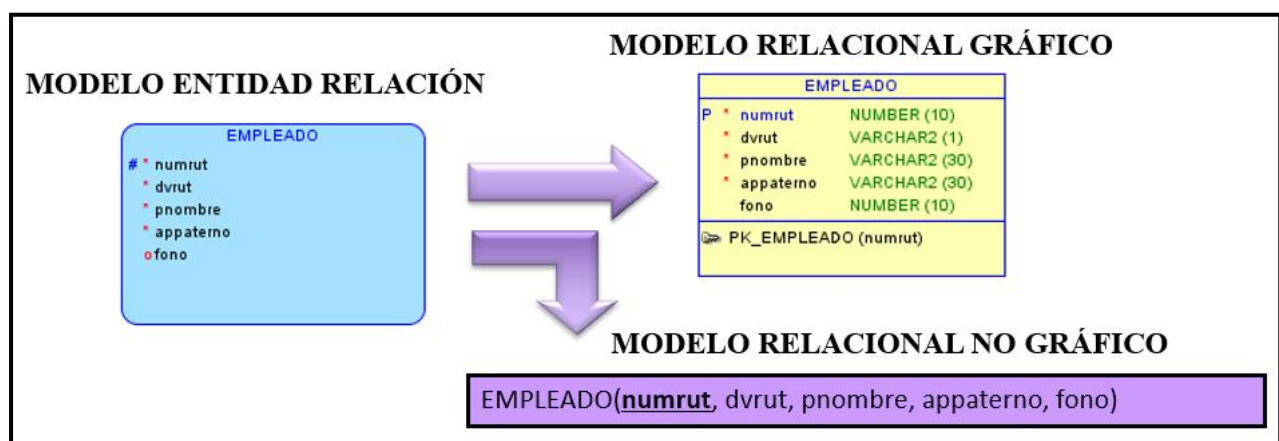
Aceptar Aplicar Reglas de Nomenclatura Cancelar Ayuda

Transformación del Modelo Entidad Relación Normalizado al Modelo Relacional

Transformación de Entidad-Fuerte

- Toda Entidad del Modelo E/R se transforma en Relación o Tabla en el Modelo Relacional.
- En la forma no gráfica del Modelo Relacional, se le debe colocar el nombre de la Relación (que debería ser el mismo nombre de la Entidad).
- Los atributos de las entidades del Modelo E/R se transforman en atributos o columnas de la Relación en el Modelo Relacional.
- En la forma no gráfica del Modelo Relacional, los atributos se colocan entre paréntesis a continuación del nombre de la Relación y separados por coma.
- Los atributos Identificadores Únicos de las Entidades del Modelo E/R se transforman en la Clave Primaria (PK) de la Relación del Modelo Relacional.
- Por lo general el formato del nombre de la clave primaria es: PK_nombretabla. En la forma no gráfica la(s) columna(s) que son parte de la clave primaria de la Relación se debe(n) subrayar.

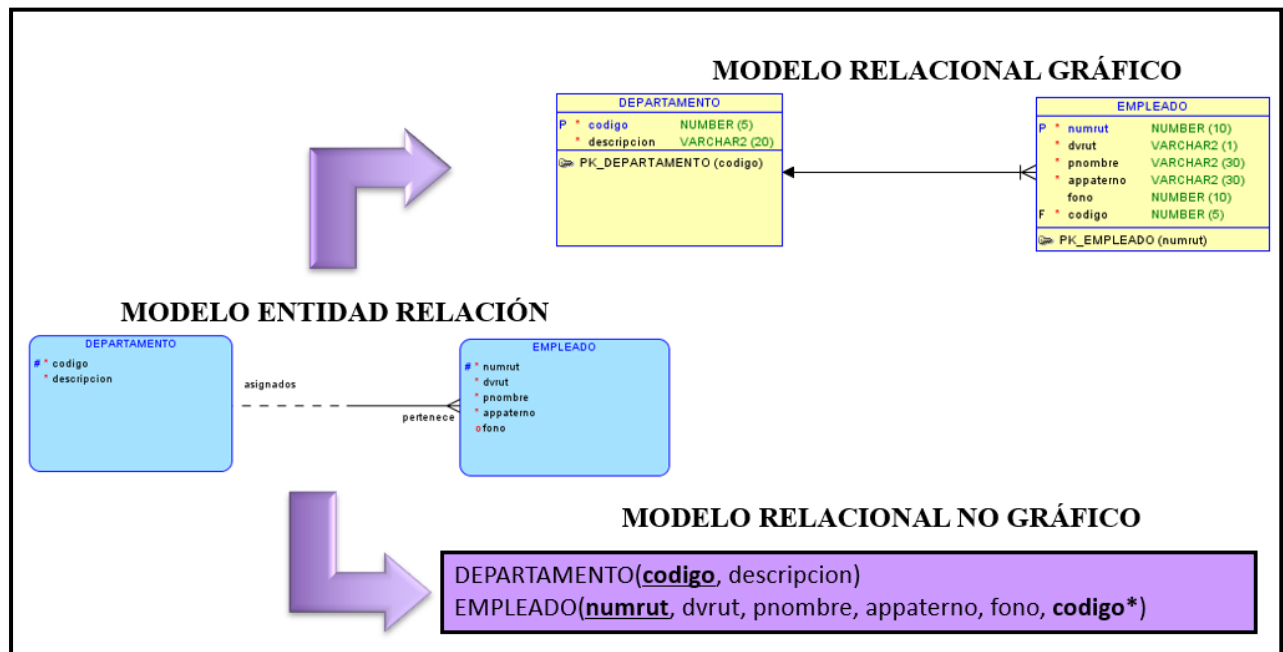
En el Ejemplo, la entidad EMPLEADO del Modelo E/R se transforma en la Relación EMPLEADO en el Modelo Relacional. El atributo identificador único de la entidad se transforma en la Clave Primaria de la Relación (marcada con una P en el Modelo Relacional gráfico).



Transformación de las relaciones 1:N

- La Relación del lado muchos (tabla relacionada) incluye como clave externa o foránea (FK) las columnas que forman la clave primaria de la Relación del lado uno (relación principal). A esta clave foránea se le debe asignar un nombre.
- Por lo general el formato del nombre de la clave foránea es: FK_tablarelacionada_tablaprincipal.
- En la forma gráfica del Modelo Relacional, la punta de la flecha indica la tabla principal que contiene la(s) columna(s) de clave primaria que con referenciadas por la tabla relacionada. La columna clave foráneas además aparece marcada con una F.
- En la forma no gráfica, la(s) columna(s) que conforman la clave foránea se marcan con un asterisco *.

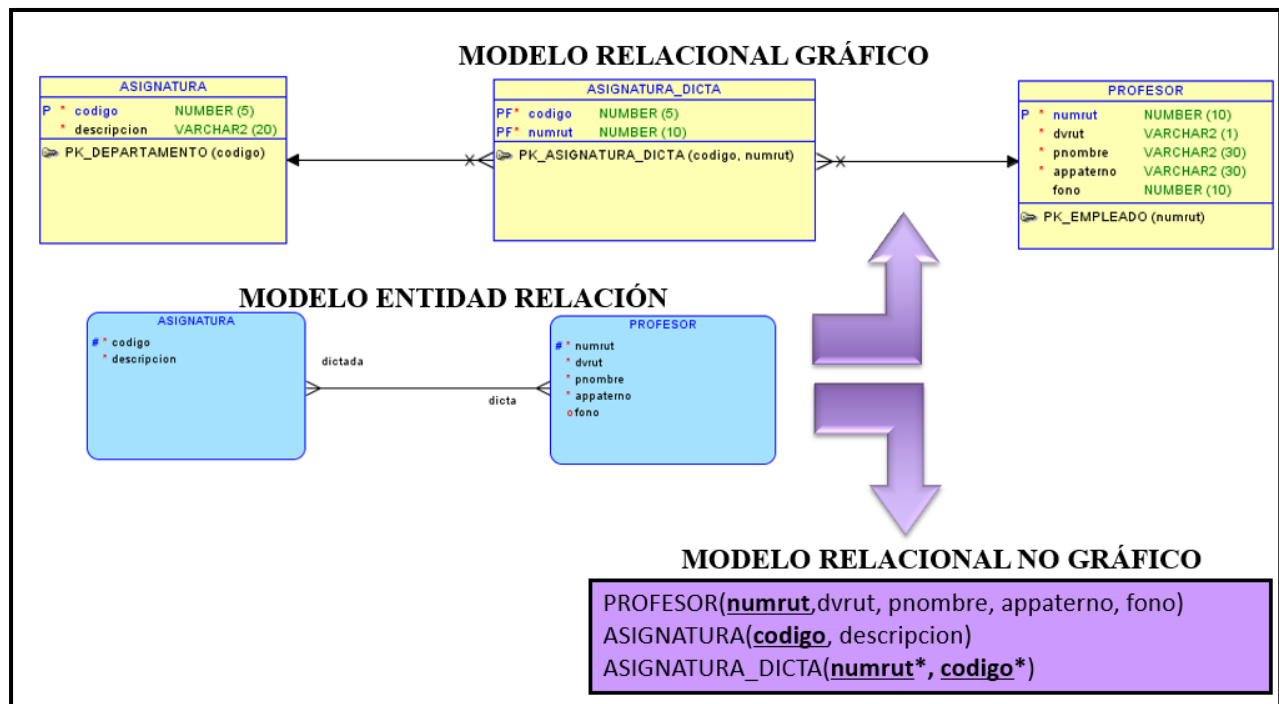
En el ejemplo, la columna codigo de la relación DEPARTAMENTO es incluida como columna de la Relación EMPLEADO y además es Clave Foránea.



Transformación de las relaciones M:N

- Para las relaciones Muchos a Muchos del modelo E/R que no pudieron ser eliminadas se debe generar una Relación Intermedia en el Modelo Relacional a la cual se le debe asignar un nombre adecuado.
- La Relación de intersección o intermedia contiene dos claves externas (claves foráneas), cada una referencia a la clave primaria de las tablas originales. En la forma no gráfica, la(s) columna(s) que conforman la clave foránea se marcan con un asterisco *.
- Ambas claves externas (foráneas) además componen la clave primaria de la Relación intermedia a la cual se le debe asignar un nombre adecuado. En la forma no gráfica se deben subrayar.

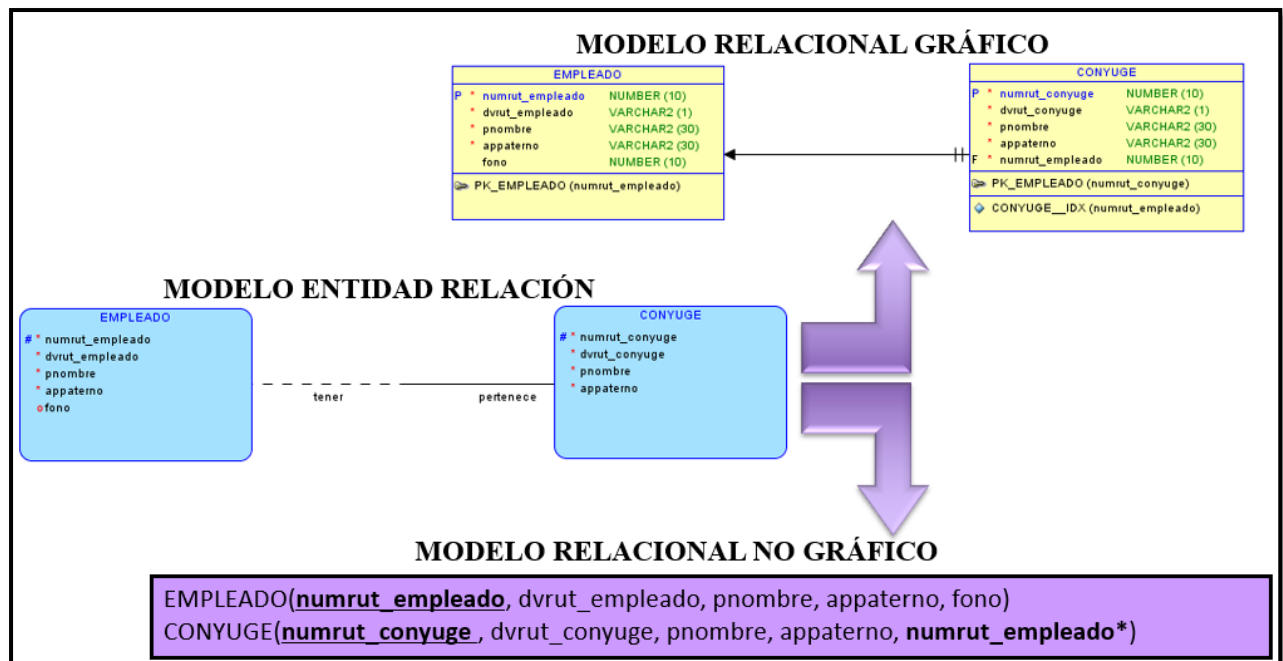
En el ejemplo, se crea la Relación intermedia ASIGNATURA_DICTA con las columnas de claves primarias de las Relaciones ASIGNATURA y PROFESOR (ambas columnas son PK y FK).



Transformación de las relaciones 1:1

- Las relaciones uno a uno en el Modelo E/R son poco comunes. En muchos casos puede implicar que las dos entidades son en realidad una sola y se deben unir.
- Si se debe mantener la relación 1:1, se coloca como clave externa o foránea en una de las Relaciones del Modelo Relacional la(s) columna(s) que conforman la clave primaria de la otra Relación (en principio daría igual en cuál de las Relaciones).

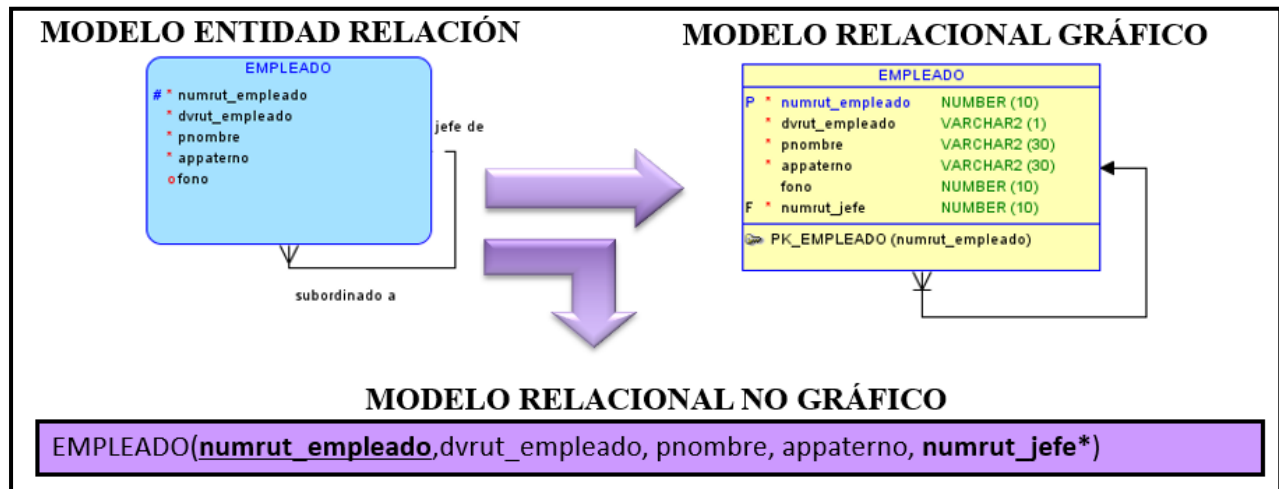
En el ejemplo, la relación del EMPLEADO con CONYUGE al transformar al Modelo Relacional la clave primaria de EMPLEADO se convierte en una columna más de CONYUGE y además es clave foránea.



Transformación de las relaciones recursivas

- Las relaciones recursivas se tratan de la misma forma que las otras, sólo que el atributo identificador único puede figurar dos veces en una Relación como resultado de la transformación y además es clave foránea. Por ello, se le debe asignar otro nombre a esa columna.

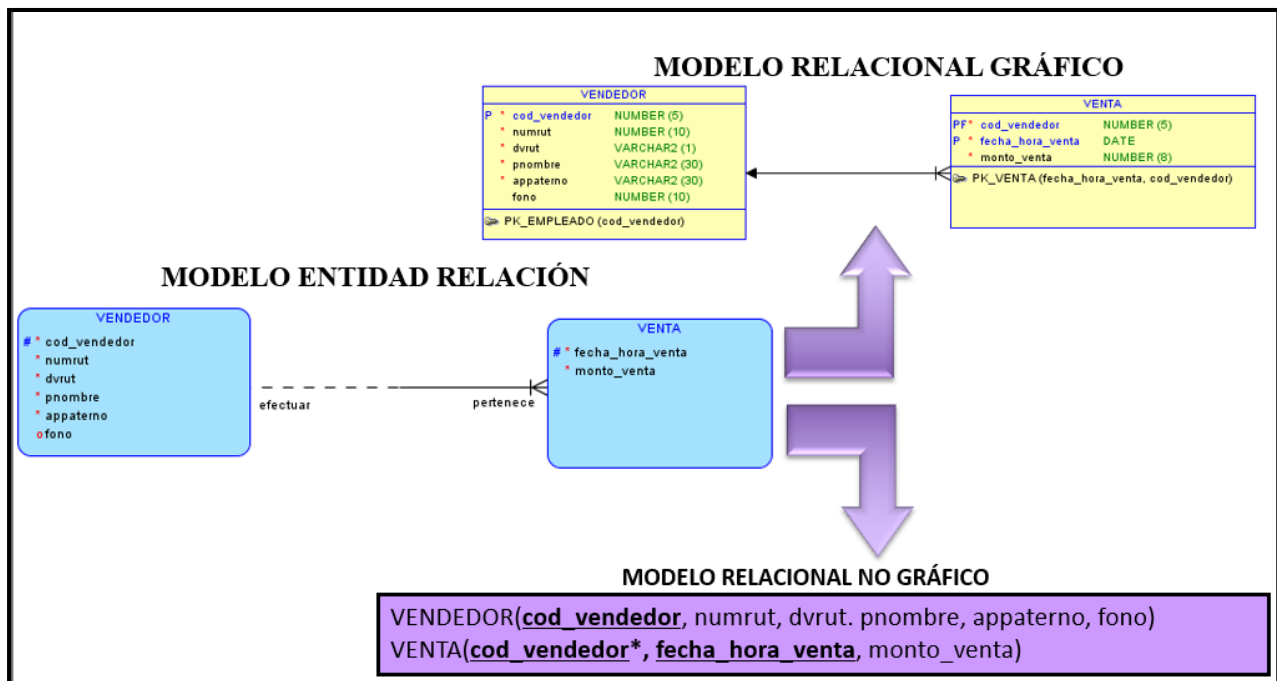
En el ejemplo, el atributo rut se transformará en otra columna de la Relación EMPLEADO en el Modelo Relacional y además será clave foránea. Por lo tanto, se le asigna el nombre de numrut_jefe.



Transformación de Entidades Débiles

- Toda entidad débil incorpora una relación implícita con una entidad fuerte.
- Se debe incorporar la clave primaria de la Relación que representa a la entidad fuerte como clave externa o foránea en la Relación que representa la entidad débil. Es más, normalmente esa clave externa además forma parte de la clave principal de la Relación que representa a la entidad débil.
- En ocasiones el identificador de la entidad débil es suficiente para identificar los ejemplares de dicha entidad, entonces ese identificador quedaría como clave principal, pero el identificador de la entidad fuerte seguiría figurando como clave externa en la entidad débil.

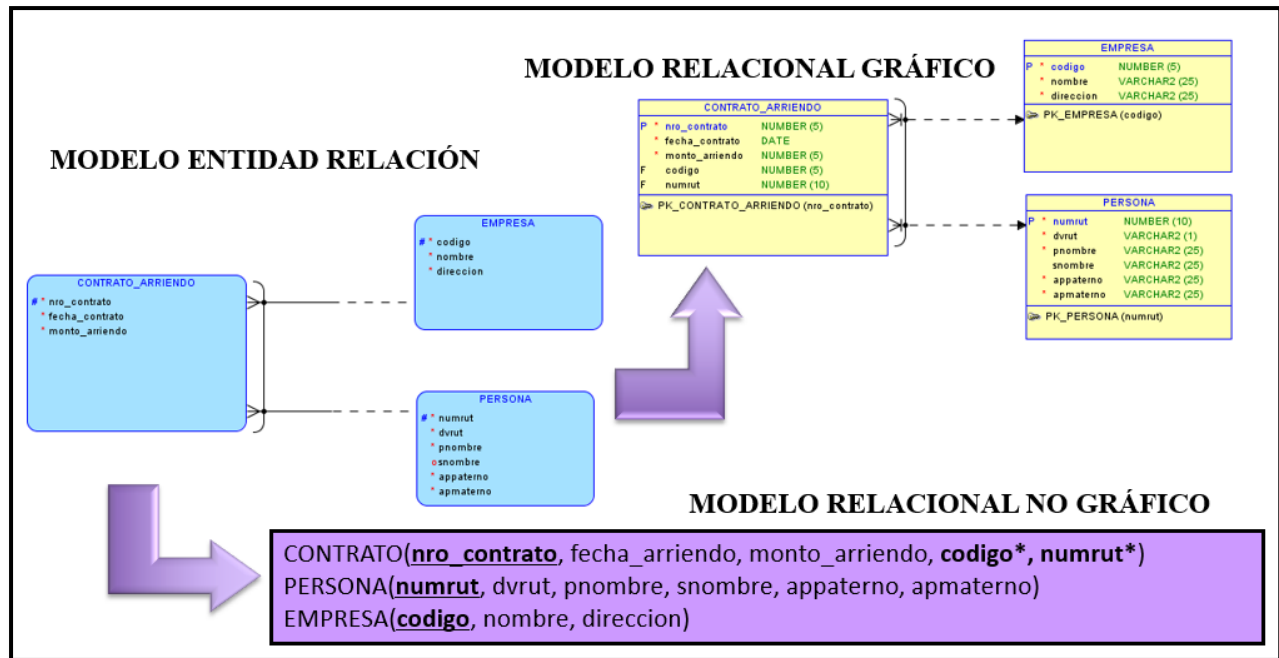
En el ejemplo, la columna `codigo_vendedor` que es la clave primaria de la Relación `VENDEDOR` se transforma en una columna clave foránea de la Relación `VENTA`. Además, junto con la columna `fecha_hora_venta` forman la clave primaria para identificar en forma única una venta de un vendedor en particular.



Transformación de Relaciones de Exclusividad

- Los arcos representan una columna de clave foránea en la Relación principal.
- El Diseño de Arco explícito crea una columna clave foránea para cada relación incluida en el arco.

En el ejemplo de la siguiente página, la columna código de la Relación EMPRESA y la columna numrut de la Relación PERSONA se transforman en columnas de claves foráneas de la Relación CONTRATO.



Transformación de Relaciones de Jerarquías

Para las Relaciones de Jerarquías (Supertipos/Subtipos) del Modelo E-R existen tres formas de poder efectuar la transformación al Modelo Relacional:

1. Diseño de los subtipos en una sola Relación junto al supertipo: es recomendable cuando los atributos de los subtipos son muy pocos.
2. Diseño de los subtipos en Relaciones separadas: cuando los atributos específicos de cada subtipo son demasiados en comparación con los atributos comunes definidos en el supertipo.
3. Diseño del supertipo y los subtipos en Relaciones separadas: en este caso cada entidad se transforma en una Relación por separado.

1. Diseño de los subtipos y supertipo en una sola Relación.

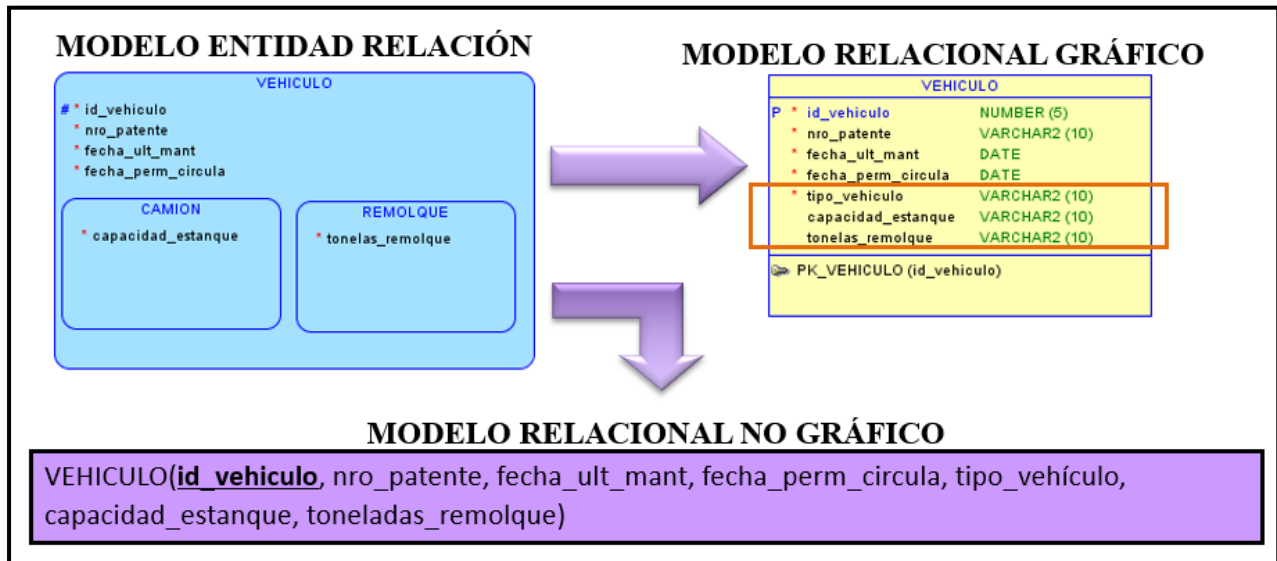
Consiste en diseñar una tabla para el supertipo con toda la información de los subtipos. En otras palabras, la tabla resultante contiene las instancias de todos los subtipos.

Es apropiado cuando los subtipos tienen pocos atributos y la consulta de los datos suele incluir datos de distintos subtipos.

Los pasos para este diseño son:

- a. Crear una Relación o tabla para el Supertipo.
- b. Crear una columna “tipo” para identificar a cuál subtipo pertenece la fila o tupla.
- c. Crear una columna para cada uno de los atributos del supertipo.
- d. Crear una columna para cada uno de los atributos de los subtipos.
- e. Crear columnas claves foráneas para cada una de las relaciones del supertipo.
- f. Crear columnas claves foráneas para cada una de las relaciones de los subtipos.

En el ejemplo, se crea la Relación VEHICULO con los atributos del supertipo y subtipos. Además, se crea la columna **tipo_vehiculo** para poder saber si la información que se almacenará será de un camión o remolque.



2. Diseño de los Subtipos en Relaciones separadas

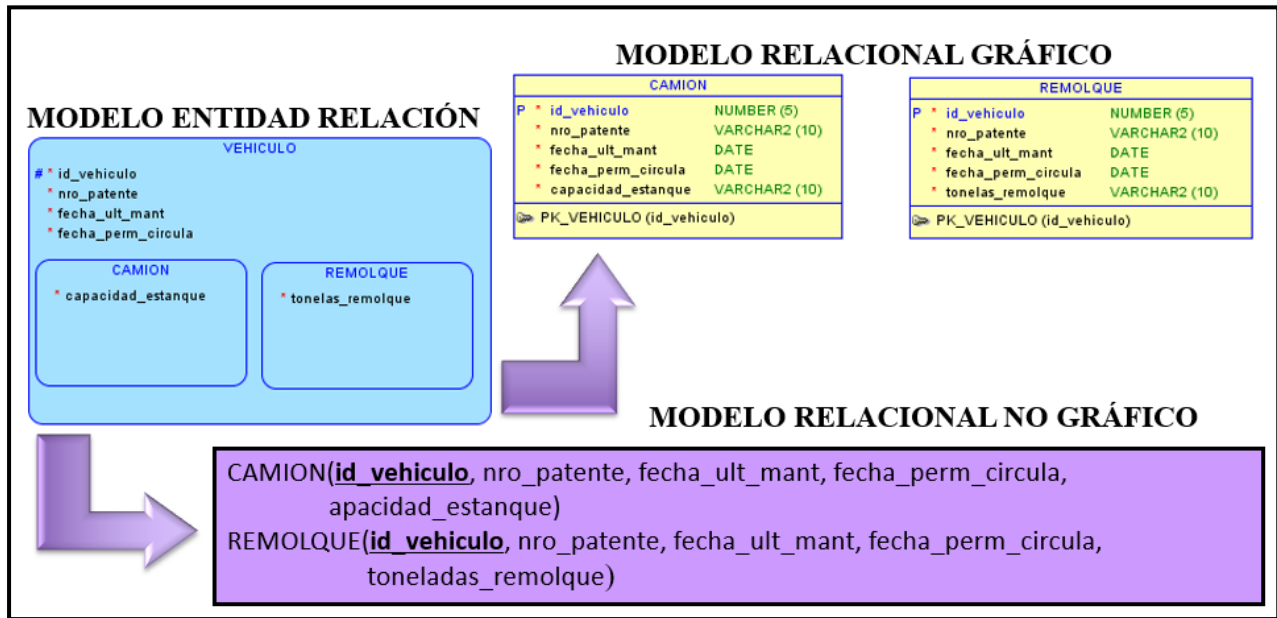
Consiste en convertir los subtipos en Relaciones diferentes, una para cada subtipo. Cada tabla tendrá instancias solo de ese subtipo.

Se usa este diseño cuando los subtipos tienen muchos atributos y relaciones específicas.

Los pasos para este diseño son:

- Crear una Relación o tabla para cada subtipo.
- En cada tabla de un subtipo crear las columnas para los atributos del supertipo.
- En cada tabla de un subtipo crear columnas claves foráneas para cada relación del subtipo.
- En cada tabla de un subtipo crear columnas claves foráneas para cada relación del supertipo.

En el ejemplo, se crea las Relaciones CAMION y REMOLQUE cada una con sus atributos y además con los atributos del supertipo incluyendo su clave primaria.



3. Diseño del supertipo y los subtipos en Relaciones separadas

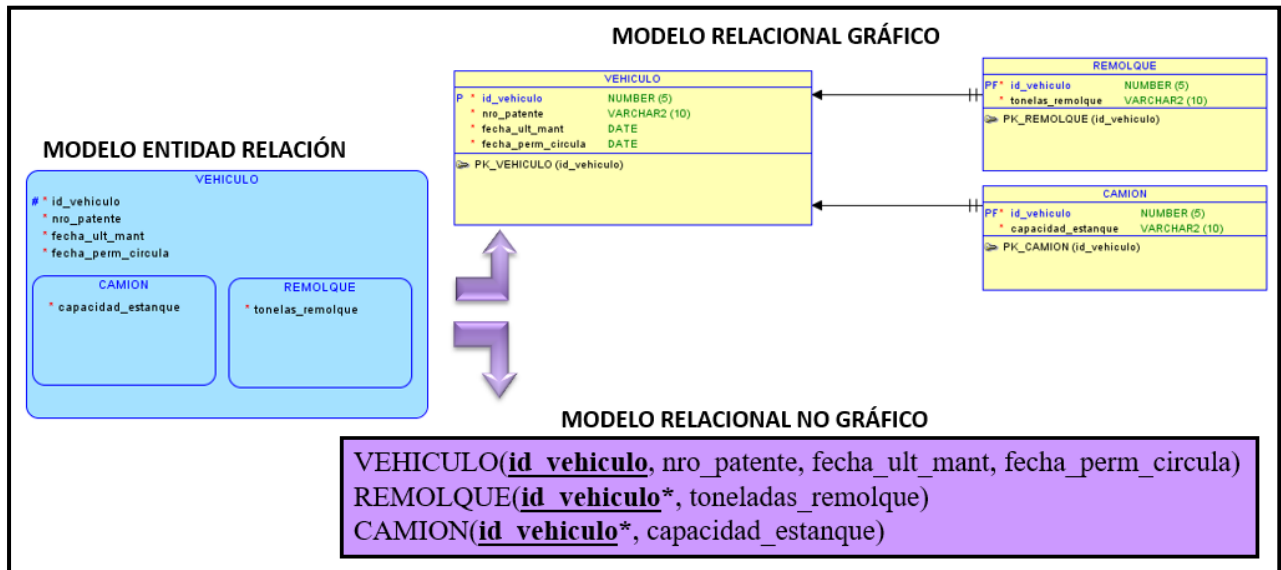
Consiste en convertir el supertipo y sus subtipos asociados en tablas diferentes.

Cada tabla tendrá como columnas los atributos de cada entidad.

Los pasos para este diseño son:

- Crear una tabla para el supertipo con sus propias columnas y clave principal.
- En cada tabla de los subtipos crear las columnas para los atributos que cada entidad del subtipo posee.
- En cada tabla de los subtipos las columnas de la clave principal del supertipo se crean como clave externa o foránea y además como clave primaria.

En el ejemplo, se crean las Relaciones VEHICULO, CAMION y REMOLQUE cada uno con sus propios atributos. Los subtipos además hereden los atributos de clave primaria del supertipo que además se transforma en clave foránea.



ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

¿Cuál es la relevancia que poseen las transformaciones de relaciones de jerarquía en el modelo de E-R?



Construcción del Modelo Relacional Gráfico y No Gráfico

Caso 1

La cadena internacional de hoteles “LUXURY HOTELES” desea informatizar la atención de sus clientes. Para ello, se desea que Ud. diseñe una Base de Datos para almacenar y gestionar la información empleada por el Hotel considerando los requerimientos planteados.

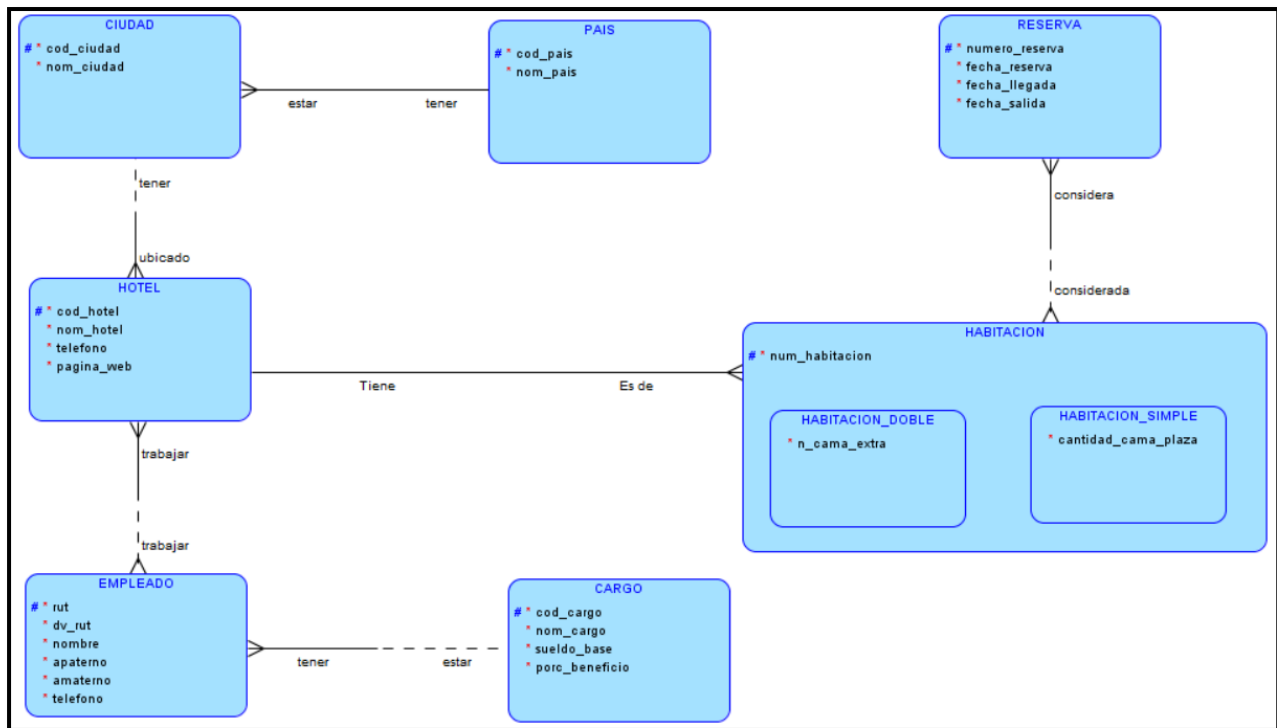
De los hoteles se sabe su código, la ciudad y el país en que se encuentra cada hotel. Además, se conoce su nombre (que es único dentro de cada ciudad), todos sus teléfonos y su dirección de página web.

Cada hotel tiene un conjunto de habitaciones con un número que las identifica dentro del mismo. Existen dos tipos de habitaciones. Las habitaciones dobles, que tienen al menos una cama matrimonial y las habitaciones simples que tienen 1 o 2 camas de una plaza. De las habitaciones dobles se sabe que algunas tienen una cama de una plaza extra, en este caso, interesa saber cuáles. De las simples, interesa registrar cuantas camas de una plaza hay en cada habitación.

Las reservas de habitaciones que realizan los clientes son únicas y se les asignan un número automático interno. Interesa registrar la fecha de la reserva, fecha de llegada y salida. Una reserva considera una o varias habitaciones.

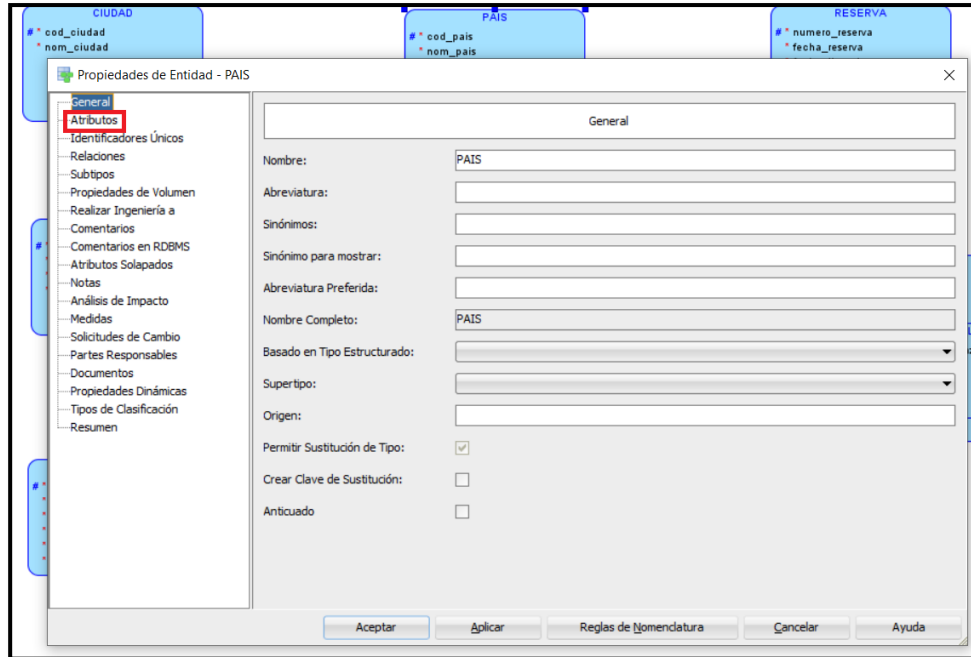
Los empleados que trabajan en los hoteles pueden trabajar en uno o varios hoteles de la cadena en jornadas diferentes. De ellos se conoce su rut, su nombre, un teléfono y el tipo de empleado según el puesto que desempeñe. De acuerdo con esto es el valor de su salario base y el porcentaje de beneficio.

Modelo Entidad Relación Normalizado:

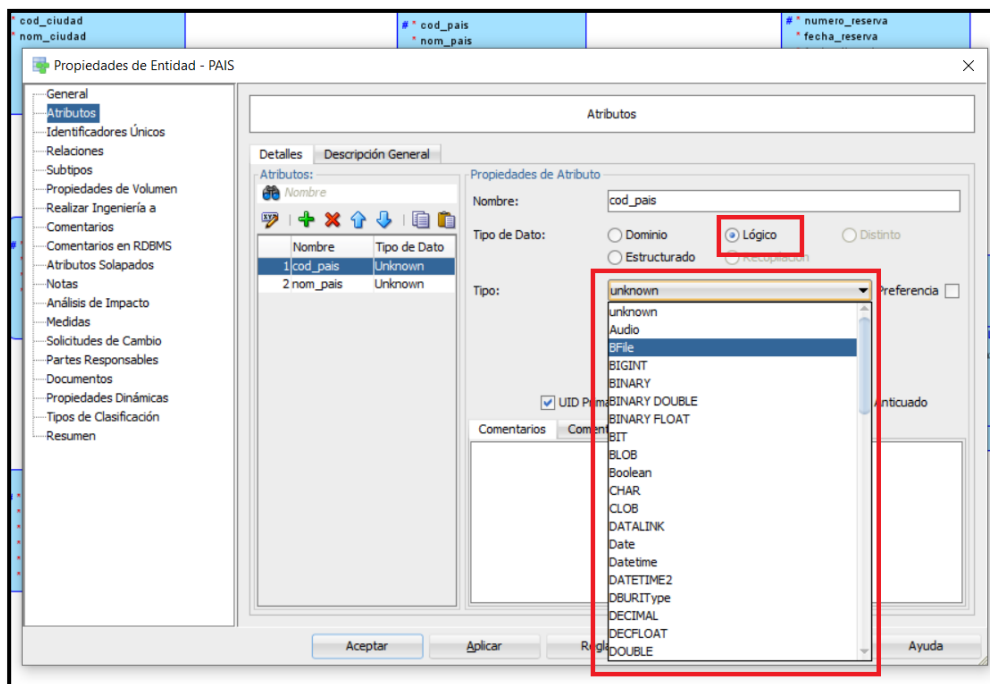


Solución Caso 1 Modelo Relacional Gráfico

Asignaremos los tipos de datos de cada atributo en cada entidad del modelo. Para ello entramos a las propiedades de la entidad y vamos a los atributos:



Luego, marcamos la opción “Lógico” y en “Tipo” buscamos el tipo de datos adecuado para cada atributo. Por lo general, usamos Varchar para texto, Numeric para números enteros y flotantes y Date para fechas.



Una vez hayamos ingresado los tipos de datos de los atributos de todas las entidades, vamos a “Realizar Ingeniería a Modelo Relacional”:

Realizar Ingeniería a Modelo Relacional

Vista de Árbol Vista Tabular

Logical Filtro

Relational_1 ☐ Como Subvista

Logical

- Entidades
- Jerarquías de Entidad
- Relaciones
- Vistas
- Subvistas

Relational_1

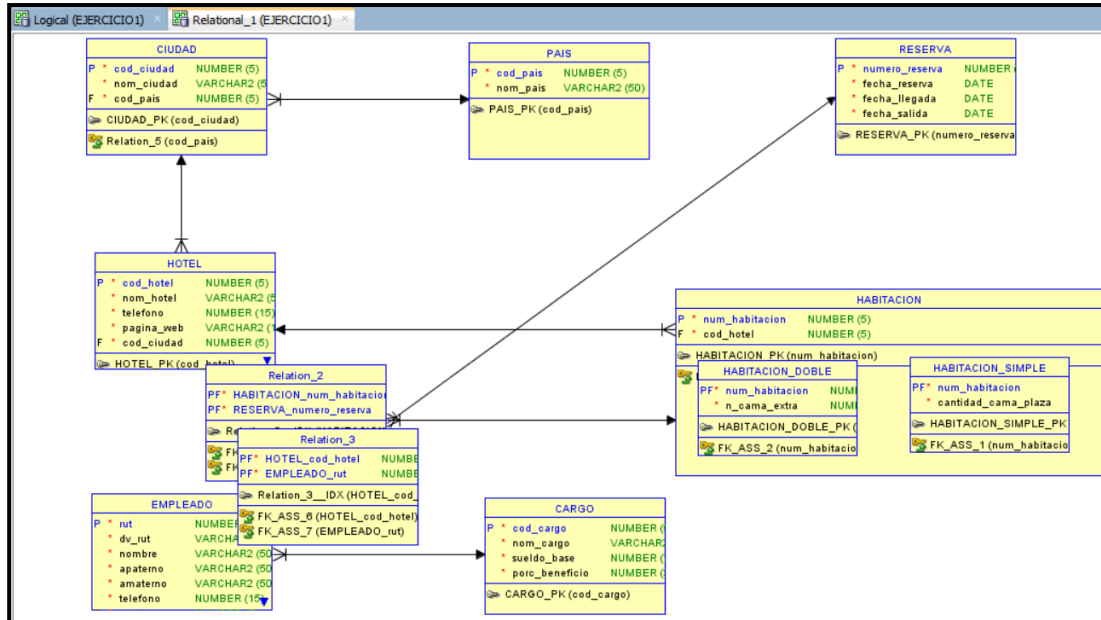
- Tablas
- Tablas Asignadas a Jerarquías
- Objetos Asignados a Relaciones
- Vistas
- Subvistas

Detalles Opciones Generales Opciones de Comparación/Copia Sincronización de Objetos Suprimidos Claves de Superposición y Desdoblamiento

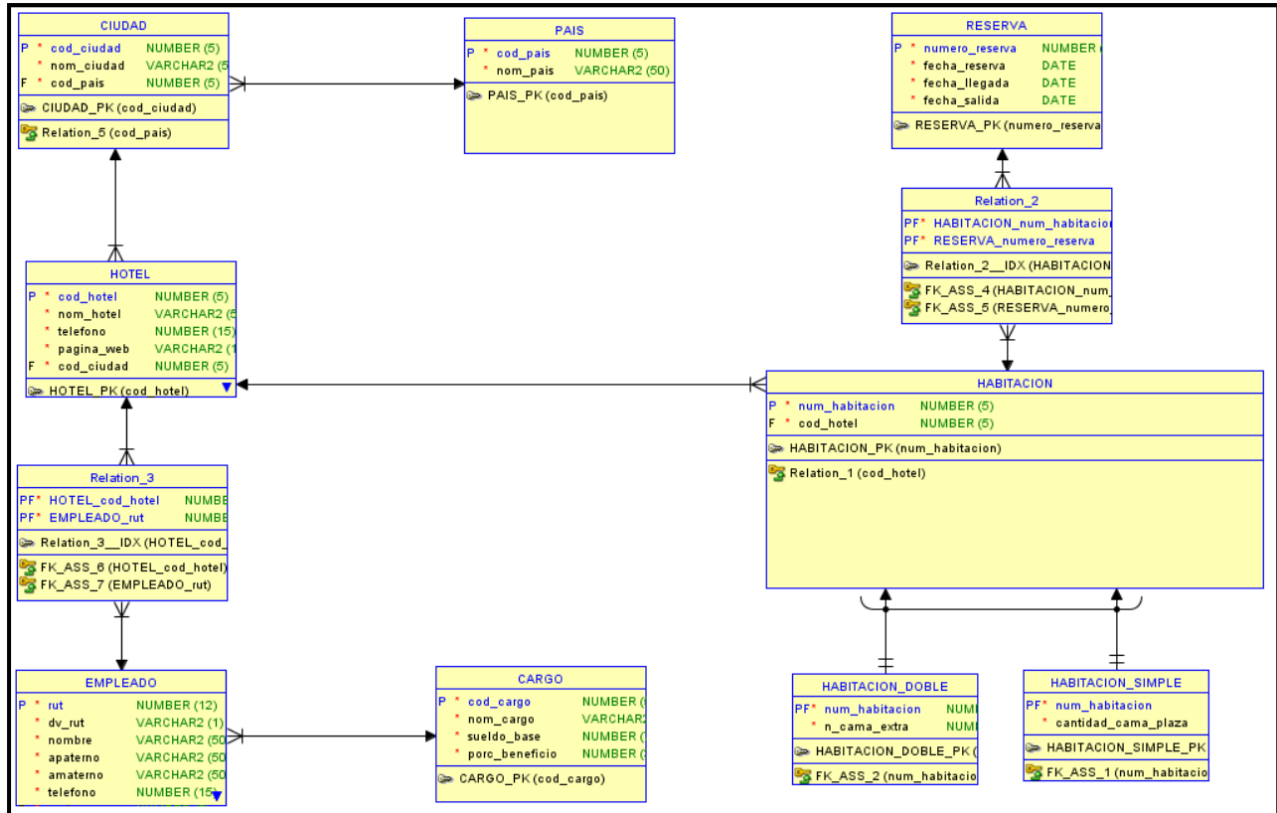
Propiedad Selección

Realizar Ingeniería Aplicar Selección Cancelar Ayuda

Se abrirá una nueva ventana: Modelo Relacional:



Lo primero será ordenar las entidades, para comenzar a ingresar nombres para relaciones intermedias (generadas por las relaciones M:N), nombres de PK (claves primarias) y claves FK (claves foráneas).

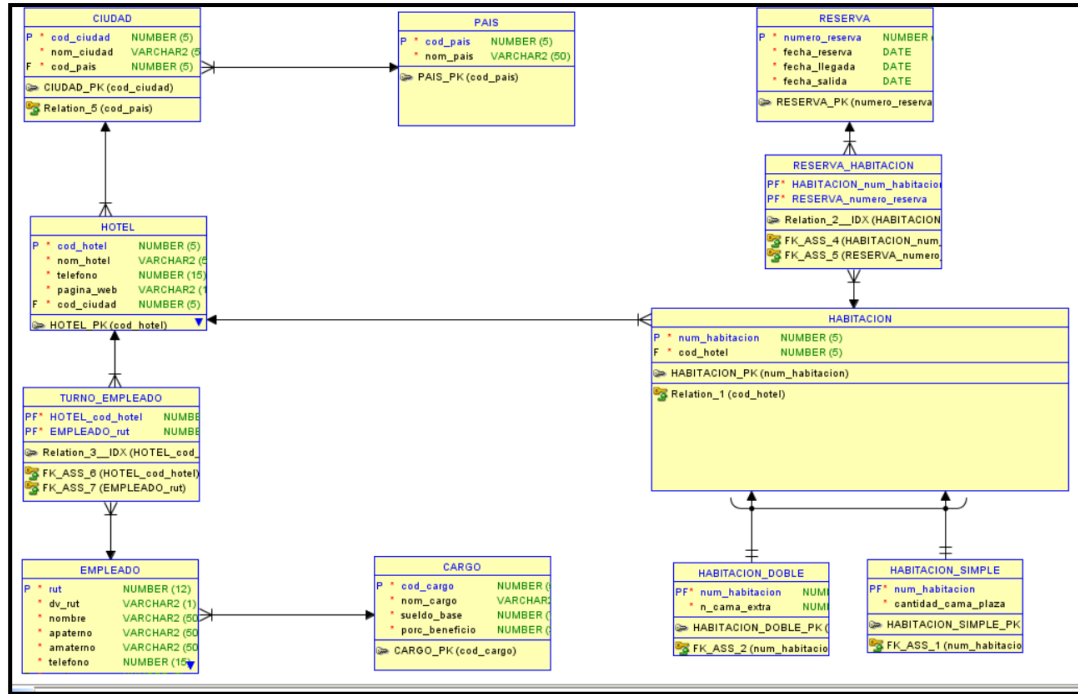


Si nos fijamos entre las tablas RESERVA y HABITACIÓN, se generó una nueva tabla, una tabla intermedia, producto de la transformación de la relación M:N que existía en el Modelo Relacional. Lo mismo sucede entre las tablas HOTEL y EMPLEADO.

Lo primero que haremos será darles un nombre apropiado a dichas tablas. En el caso de la tabla entre RESERVA y HABITACION, podríamos nombrarla justamente como RESERVA_HABITACION.

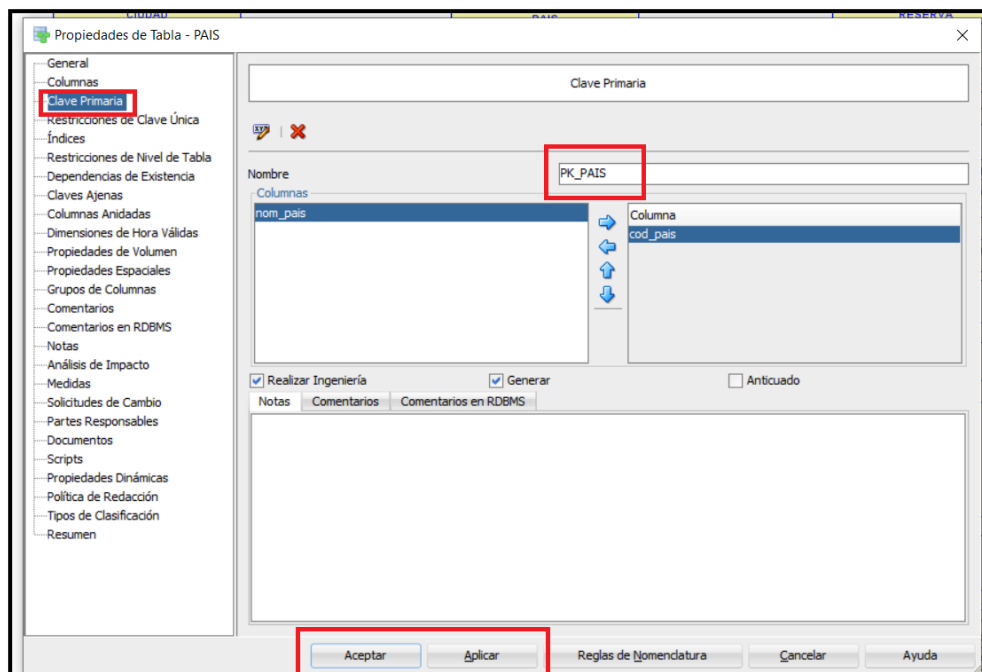
En el caso de la tabla entre HOTEL y EMPLEADO podría ser llamada TURNO_EMPLEADO.

Para cambiar el nombre de una tabla, se realiza de la misma manera que con las entidades, es decir, entrando a las propiedades de esta.

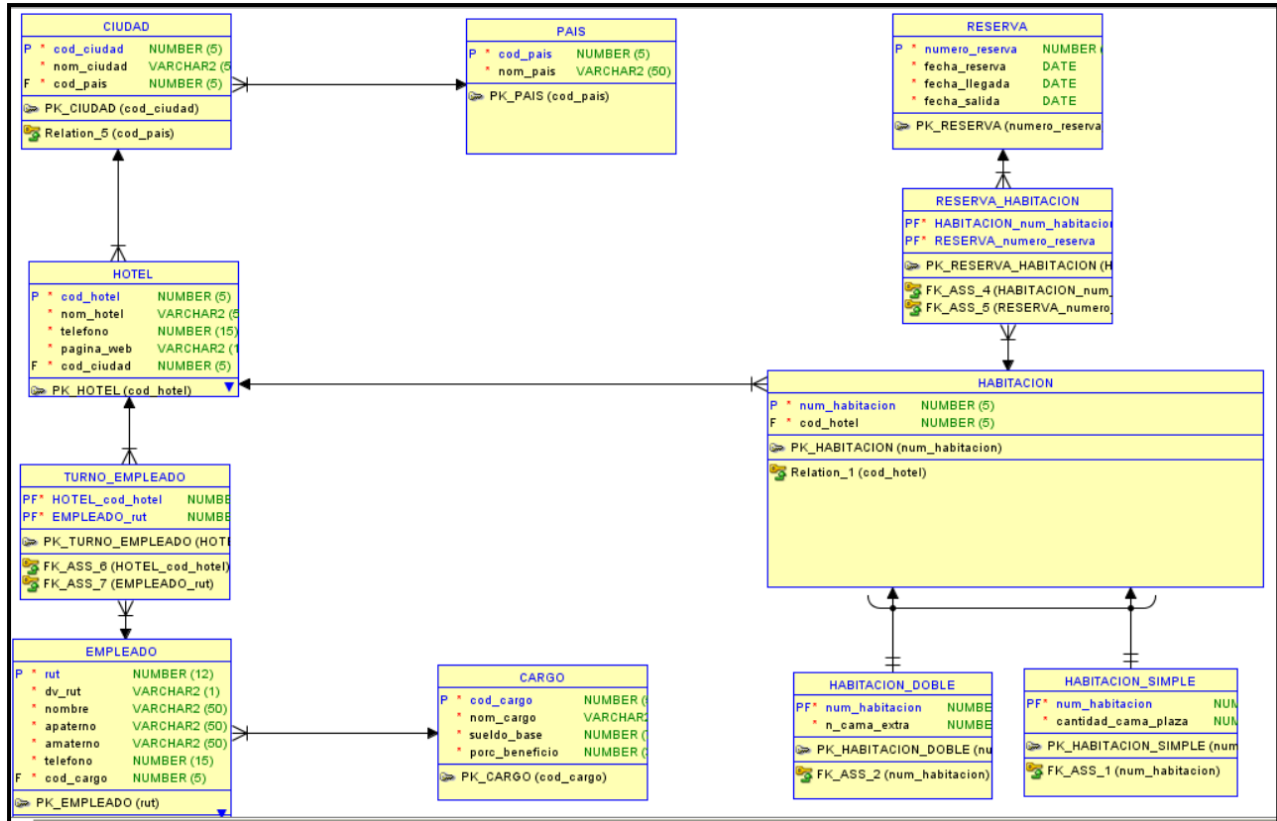


A continuación, asignaremos a cada una de las entidades un nombre apropiado para las claves primarias (PK): Recordemos que una buena práctica es que las claves primarias de las tablas sean llamadas: PK_NOMBRE_TABLA. Por ejemplo: PK_PAIS.

Para cambiar el nombre de las PK, entramos a las propiedades de la tabla y vamos a la opción: Clave Primaria. Y renombramos la PK.



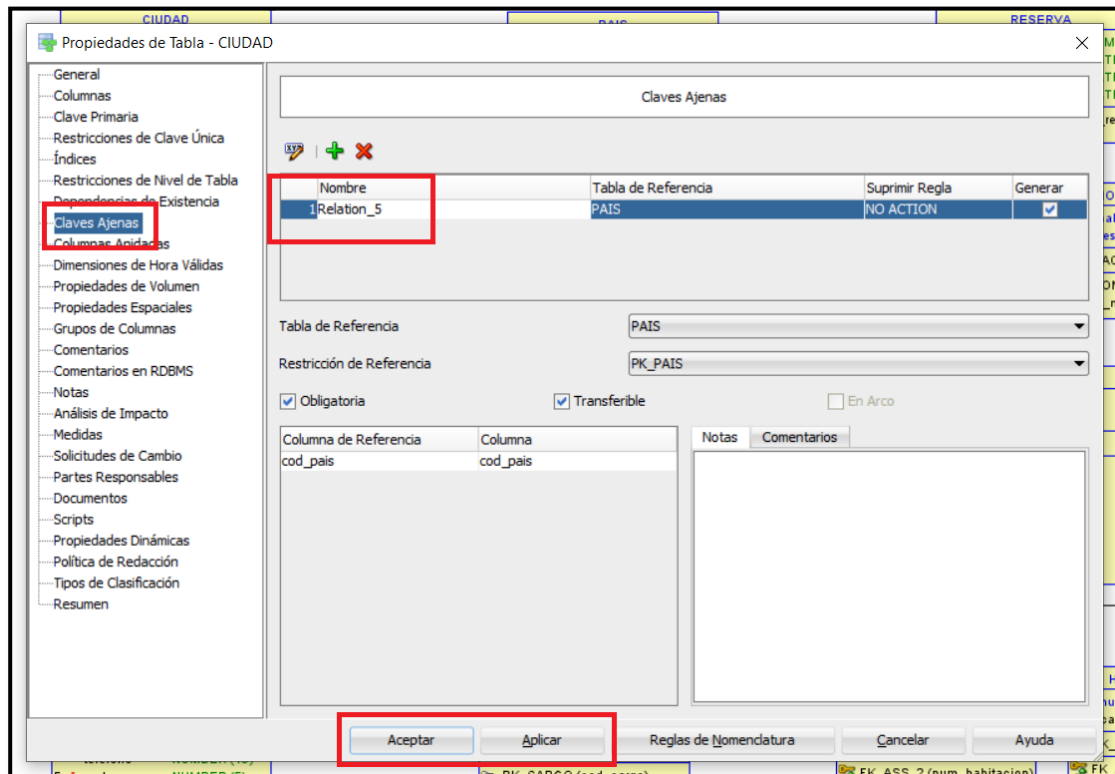
Replicamos estos pasos en todas las entidades:



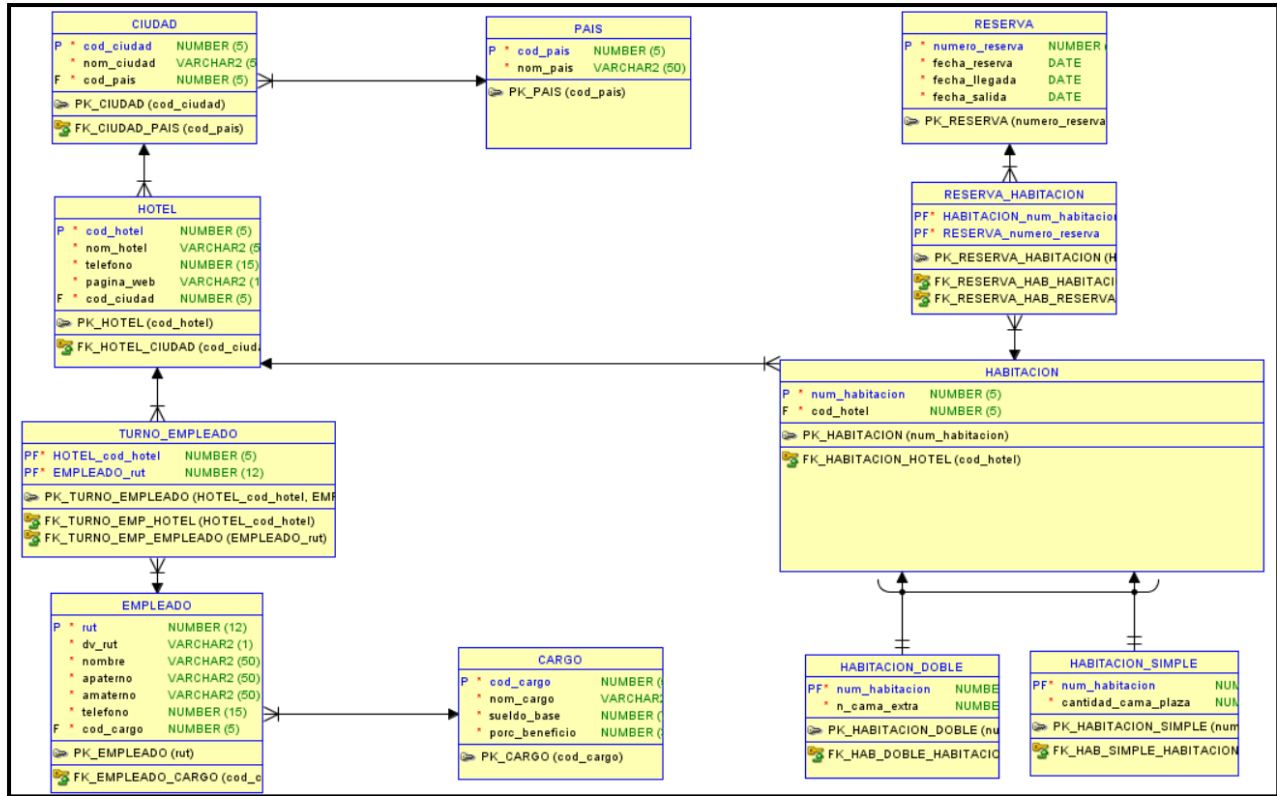
Por último, renombramos las claves foráneas (FK) en aquellas entidades que posean. El nombre apropiado es: FK_tablarelacionada_tablaprincipal. Ejemplo: En el caso de la tabla CIUDAD, su clave foránea (proveniente de la tabla PAIS) sería: FK_CIUDAD_PAIS.

Para cambiar el nombre de las FK, entramos a las propiedades de la tabla y vamos a la opción: Claves Ajenas. Y renombramos la FK. En el caso que una tabla posea más de una clave foránea, por las relaciones con otras tablas, repetimos el proceso en cada clave foránea.

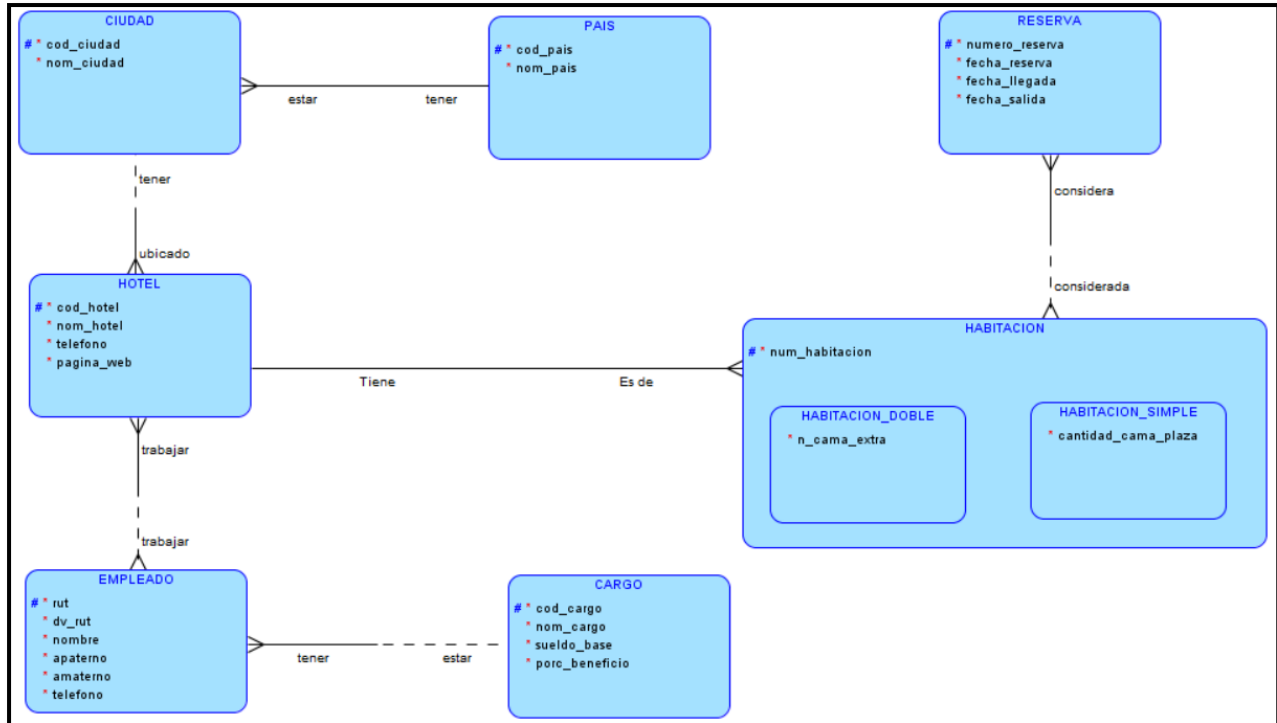
** Hay que recordar que los nombres de las tablas, de los atributos, claves primarias y claves foráneas NO pueden superar los 30 caracteres, por tanto, en caso de ser necesario, se puede abreviar el nombre.



Repetimos estos pasos en todo el Modelo Relacional:



Modelo Entidad Relación Normalizado:



Solución Caso 1 Modelo Relacional No Gráfico

Aplicaremos las reglas de transformación del Modelo Entidad Relación al Modelo Relacional en forma manual y escrita.

No importa el orden o sentido en el cual se comience a realizar la transformación, pero siguiendo las recomendaciones en informática: Lo ideal es hacerlo de izquierda a derecha y de arriba hacia abajo (ejecución secuencial).

Comenzaremos con la entidad CIUDAD. La regla de transformación de entidades fuertes señala:

- Toda Entidad del Modelo E/R se transforma en Relación o Tabla en el Modelo Relacional.
- En la forma no gráfica del Modelo Relacional, se le debe colocar el nombre de la Relación (que debería ser el mismo nombre de la Entidad).
- En la forma no gráfica del Modelo Relacional, los atributos se colocan entre paréntesis a continuación del nombre de la Relación y separados por coma.
- En la forma no gráfica la(s) columna(s) que son parte de la clave primaria de la Relación se debe(n) subrayar.

Por tanto:

CIUDAD (cod_ciudad, nom_ciudad)

Ahora, iremos por la entidad PAIS:

CIUDAD (cod_ciudad, nom_ciudad)

PAIS (cod_pais, nom_pais)

Por último, resolvemos la relación entre ellos, que es una relación 1:N. La regla de transformación de las relaciones 1:N señala que:

- La Relación del lado muchos (tabla relacionada) incluye como clave externa o foránea (FK) las columnas que forman la clave primaria de la Relación del lado uno (relación principal). A esta clave foránea se le debe asignar un nombre.
- Por lo general el formato del nombre de la clave foránea es: FK_tablarelacionada_tablaprincipal.
- En la forma no gráfica, la(s) columna(s) que conforman la clave foránea se marcan con un asterisco *.

Por tanto, quedaría:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

A continuación, representaremos a la entidad HOTEL

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web)

Transformamos la relación entre CIUDAD y HOTEL. Nuevamente una relación 1:N

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

Representamos la entidad EMPLEADO:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono)

Ahora, transformamos la relación M:N que existe entre HOTEL y EMPLEADO. La regla de transformación de las relaciones M:N señala:

- Para las relaciones Muchos a Muchos del modelo E/R que no pudieron ser eliminadas se debe generar una Relación Intermedia en el Modelo Relacional a la cual se le debe asignar un nombre adecuado.
- La Relación de intersección o intermedia contiene dos claves externas (claves foráneas), cada una referencia a la clave primaria de las tablas originales. En la forma no gráfica, la(s) columna(s) que conforman la clave foránea se marcan con un asterisco *.
- Ambas claves externas (foráneas) además componen la clave primaria de la Relación intermedia a la cual se le debe asignar un nombre adecuado. En la forma no gráfica se deben subrayar.

Por tanto, el Modelo Relacional quedaría:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono)

TURNO_EMPLEADO (cod_hotel*, rut_empleado*)

Representamos la entidad CARGO:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono)

TURNO_EMPLEADO (cod_hotel*, rut_empleado*)

CARGO (cod_cargo, nom_cargo, sueldo_base, porc_beneficio)

Transformamos la relación entre EMPLEADO y CARGO. Es una relación 1:N. Por tanto:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono, cod_cargo*)

TURNO_EMPLEADO (cod_hotel*, rut_empleado*)

CARGO (cod_cargo, nom_cargo, sueldo_base, porc_beneficio)

Ahora, representamos el lado derecho, de arriba hacia abajo. Por tanto, comenzamos representando la entidad RESERVA:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono, cod_cargo*)

TURNO_EMPLEADO (cod_hotel*, rut_empleado*)

CARGO (cod_cargo, nom_cargo, sueldo_base, porc_beneficio)

RESERVA (numero_reserva, fecha_reserva, fecha_llegada, fecha_salida)

A continuación, representamos la entidad HABITACION:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono, cod_cargo*)

TURNO_EMPLEADO (cod_hotel*, rut_empleado*)

CARGO (cod_cargo, nom_cargo, sueldo_base, porc_beneficio)

RESERVA (numero_reserva, fecha_reserva, fecha_llegada, fecha_salida)

HABITACION (num_habitacion)

Transformamos la relación existente entre RESERVA y HABITACION. Es una relación M:N, ya sabemos cómo debemos transformar:

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono, cod_cargo*)

TURNO_EMPLEADO (cod_hotel*, rut_empleado*)

CARGO (cod_cargo, nom_cargo, sueldo_base, porc_beneficio)

RESERVA (numero_reserva, fecha_reserva, fecha_llegada, fecha_salida)

HABITACION (num_habitacion)

RESERVA_HABITACION (numero_reserva*, num_habitacion*)

Por último, representamos las entidades HABITACION_SIMPLE y HABITACION_DOBLE. Ambas entidades son subtipos de la entidad de supertipo HABITACION, por lo que corresponde a una relación de jerarquía. La transformación de relaciones de jerarquía señala:

Para las Relaciones de Jerarquías (Supertipos/Subtipos) del Modelo E-R existen tres formas de poder efectuar la transformación al Modelo Relacional:

1. Diseño de los subtipos en una sola Relación junto al supertipo: es recomendable cuando los atributos de los subtipos son muy pocos.
2. Diseño de los subtipos en Relaciones separadas: cuando los atributos específicos de cada subtipo son demasiados en comparación con los atributos comunes definidos en el supertipo.
3. Diseño del supertipo y los subtipos en Relaciones separadas: en este caso cada entidad se transforma en una Relación por separado.

Para este caso, utilizaremos la solución 3. Es decir, crearemos la entidad de supertipo HABITACION (ya creada) y cada una de las entidades de subtipo: HABITACION_SIMPLE y HABITACION_DOBLE.

CIUDAD (cod_ciudad, nom_ciudad, cod_pais*)

PAIS (cod_pais, nom_pais)

HOTEL (cod_hotel, nom_hotel, telefono, pagina_web, cod_ciudad*)

EMPLEADO (rut, dv_rut, nombre, apaterno, amaterno, telefono, cod_cargo*)

TURNO_EMPLEADO (cod_hotel*, rut_empleado*)

CARGO (cod_cargo, nom_cargo, sueldo_base, porc_beneficio)

RESERVA (numero_reserva, fecha_reserva, fecha_llegada, fecha_salida)

HABITACION (num_habitacion)

RESERVA_HABITACION (numero_reserva*, num_habitacion*)

HABITACION_SIMPLE (num_habitacion*, cantidad_cama_plaza)

HABITACION_DOBLE (num_habitacion*, n_cama_extra)

Listo, ya tenemos el Modelo Relacional No Gráfico del Caso 1!

Conclusión

En esta semana se profundizó en torno a los conceptos del Diseño Lógico y el Modelo Relacional: sus características, sus objetivos, las 12 reglas de Codd que aseguran la correcta construcción de este modelo, entre otros elementos.

Por otro lado, se ahondó sobre la terminología que posee el Modelo Relacional y, sobre todo, acerca de cómo transformar un Modelo Entidad-Relación Normalizado en un Modelo Relacional, tanto en forma gráfica, como no gráfica.

Por último, se detalló lo relativo a la construcción o elaboración un Modelo Relacional Gráfico en la herramienta CASE SQL Data Modeler y cómo representar los conceptos anteriormente señalados (además del Modelo Relacional No Gráfico en forma escrita).

Referencias

- + Abraham Silberschatz , Henry F. Korth y S. Sudarshan. (2014). FUNDAMENTOS DE BASES DE DATOS 6ED. España: McGraw-Hill.
- + Antolin Muñoz Chaparro. (2018). Oracle 12c SQL. Curso práctico de formación. España: RC Libros.
- + Carlos Coronel, Steven Morris, Peter Rob.(2012) Bases de Datos: Diseño, Implementación y Administración. España: Cengage Learning.

